

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **G. GOWTHAMI(192524285)** Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. GOWTHAMI

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

ANSWERS

1. Emergency Medicine Drone Routing – Greedy Best-First Search vs A*

i. Greedy Best-First Search

Greedy Best-First Search selects the node with the smallest heuristic value: where $h(n)$ is the estimated distance from the current node to the destination.

Example

Suppose the drone has three possible routes:

Node

Straight-line distance $\square$

A

12 km

B

8 km

C

15 km

Greedy Best-First Search selects:

because:

The algorithm focuses only on which node appears closest to the destination.

Strengths

Very fast.

Simple to implement.

Requires relatively little computation.

Useful when an approximate answer is sufficient.

Suitable for urgent routing when decisions must be made quickly.

Weaknesses

Does not consider the cost already travelled.

Does not guarantee the shortest or safest route.

A straight-line heuristic may ignore blocked roads or severe weather.

Can choose a route that looks short but has high risk or cost.

ii. A* Search

A* combines actual cost and estimated future cost:

where:

$\square$ = actual cost from start to node $\square$

$\square$ = estimated cost from $\square$ to destination

$\square$ = estimated total route cost.

Example

Suppose:

Node

$\square$

$\square$

$\square$

A

4

10

14

B

7

5

12

C

3

12

15

A* selects:

because:

It balances past cost + estimated future cost.

Comparison

Feature

Greedy BFS

A*

Evaluation

☐

☐

Considers travelled cost

No

Yes

Speed

Usually faster

Usually slower

Optimality

Not guaranteed

Can be optimal with suitable heuristic

Safety/cost awareness

Lower

Higher

iii. Impact of Heuristic Accuracy

The heuristic has a major effect on both algorithms.

Greedy Best-First Search

Greedy search depends almost completely on ☐.

If:

the search can be very efficient.

If ☐ is inaccurate, the algorithm may select a dangerous or expensive route.

A*

A* is also affected by heuristic accuracy.

If the heuristic is admissible:

where ☐ is the actual minimum remaining cost, A* can guarantee an optimal solution under the standard conditions.

Example

Actual remaining costs:

Route A = 10

Route B = 20

If the heuristic estimates:

the estimates are reasonable.

But if:

the search may be misled.

iv. Dynamic Blocked Paths

Suppose the drone selects:

Start $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ Destination

but suddenly B becomes inaccessible.

The original search tree is no longer completely valid.

The system must:

Detect the blocked path.

Remove/invalidate the affected edge.

Update the search graph.

Recalculate route costs.

Search for an alternative path.

Repeatedly running complete A* from the beginning may be computationally expensive.

For a highly dynamic environment, incremental/replanning algorithms such as D or D Lite** can be more suitable because they are designed to efficiently update routes when the environment changes.

v. Recommended Algorithm

For this emergency drone system, I would recommend:

because the drone must consider both travel cost and estimated remaining distance.

For a rapidly changing environment, an even better practical choice is:

Justification

A* provides better cost-aware routing.

Heuristics improve search speed.

Dynamic replanning handles newly blocked paths.

Risk/weather costs can be incorporated into $\square$.

Safety is more important than simply selecting the geometrically shortest route.

2. Smart-City Traffic Signal Optimization

Problem Formulation

Let:

where each $\square$ represents the signal timing at an intersection.

For example:

The objective is to minimise:

where:

$\square$ = total waiting time

$\square$ = fuel consumption

$\square$ = traffic congestion

$\square$ = importance weights.

Example

Suppose:

and:

Then:

The objective is to minimise 4300.

i. Hill Climbing

Hill Climbing starts with an initial solution and repeatedly moves to a better neighbouring solution.

Example

Suppose lower cost is better:

Solution

Cost

Current

500

Neighbour A

450

Neighbour B

470

Neighbour C

520

The algorithm chooses:

because:

It continues until no neighbouring solution is better.

ii. Local Maxima, Plateaus and Ridges

Although the traffic problem is a minimisation problem, these concepts can be described in terms of the search landscape.

Local Optimum

The current signal timing may be better than all nearby timings, but another combination far away may be much better.

Example:

Current solution = 200

Nearby solutions = 210, 215, 205

Distant solution = 100

Hill Climbing stops at 200 even though 100 is globally better.

Plateau

Many neighbouring solutions have approximately the same objective value.

Example:

The algorithm cannot identify a clearly better direction.

Ridge

The best direction may require a sequence of changes that initially appear worse.

Hill Climbing may reject these temporary increases and fail to reach a better solution.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

Simulated Annealing sometimes accepts a worse solution to escape local optima.

Probability:

where:

ΔC = increase in cost

T = temperature.

Example

Suppose:

Then:

So there is approximately a 60.7% probability of accepting the worse solution.

As temperature decreases, the probability decreases.

Genetic Algorithm

A Genetic Algorithm maintains a population of possible traffic-signal solutions.

Steps:

Initial Population



Fitness Evaluation



Selection



Crossover



Mutation



New Population



Repeat

Example chromosome:

[45, 30, 50, 35, 40]

where each value represents a signal timing parameter.

iv. Recommended Approach

For hundreds of intersections with a continuously changing environment, I recommend a:

Why?

Genetic Algorithms:

Handle very large search spaces.

Explore many solutions simultaneously.

Reduce the risk of local-optimum trapping.

Work well with complex objective functions.

Real-time local optimisation can then quickly refine the solution.

For rapidly changing traffic, the system should repeatedly optimise using new traffic sensor data rather than relying on one fixed solution.

3. Mars Rover – Online Search

i. Why Online Search?

An offline search algorithm assumes that the environment/model is available before searching.

A Mars rover does not have a complete map.

It may initially know:

Start → ? → ? → ? → Goal

As it moves, sensors reveal:

Rocks

Craters

Slopes

Terrain conditions

Obstacles

Therefore, it must:

This is the fundamental reason an online search agent is required.

ii. Partial Observability and Dynamic Environment

The rover has limited sensing range.

Suppose the rover sees only 20 metres ahead.

It cannot know:

Current position → 20m → Unknown → Unknown → Goal

This creates partial observability.

Challenges include:

Unknown obstacles.

Incomplete terrain information.

Sensor noise.

Communication delays.

Limited computational resources.

Limited energy.

Risk of becoming trapped.

Changing environmental conditions.

iii. Internal Model

The rover maintains an internal representation of its environment.

Initially:

Unknown Unknown Unknown

Unknown Rover Unknown

Unknown Unknown Unknown

After sensing:

Safe Rock Unknown

Safe Rover Safe

Unknown Safe Crater

The rover continuously updates this model.

Process

When a new hazard is discovered, the rover updates the map and searches for another route.

iv. Comparison of Search Strategies

Strategy

Knowledge

Suitable for Mars?

BFS

No heuristic

Limited

DFS

No heuristic

Limited

Uniform Cost

Path cost

Partially

Greedy BFS

Heuristic

Useful but risky

A*

Cost + heuristic

Useful

Online Search

Learns while acting

Highly suitable

Offline A* assumes the search graph is sufficiently known.

Online search is specifically useful when the environment is unknown or incompletely known.

v. Recommended Approach

The best solution is:

A practical rover can combine:

Sensor-based environment mapping

Heuristic search

A* for known portions

Online exploration

Risk-aware cost functions

Continuous replanning

For example:

A route that is slightly longer but safer may therefore be selected.

4. University Exam Timetabling as a CSP

CSP Formulation

A Constraint Satisfaction Problem (CSP) consists of:

where:

□ = variables

□ = domains

□ = constraints.

i. Variables, Domains and Constraints

Variables

Each exam can be represented by:

For example:

X1 = Mathematics

X2 = Physics

X3 = DBMS

X4 = AI

Domains

A domain contains possible values.

For example:

Rooms:

Constraints

1. No Student Conflict

If students take Mathematics and Physics:

2. Room Capacity

3. Faculty Availability

If a faculty member is unavailable Monday morning:

4. Precedence Constraint

If Subject A must occur before Subject B:

5. Special Accommodation

A student requiring a special room must be assigned to an appropriate room.

ii. Backtracking Search

Backtracking assigns variables one at a time.

Example:

Assign Mathematics



Assign Physics



Conflict?



Yes



No



Backtrack



Continue

Suppose:

and Physics has a student conflict.

If:

violates the constraint:

then the algorithm backtracks and tries another time.

iii. Forward Checking

Forward checking removes impossible values from the domains of unassigned variables.

Example

Initially:

If Mathematics is assigned:

then forward checking removes 9AM:

This reduces future search.

Without Forward Checking

The algorithm may continue exploring invalid assignments.

With Forward Checking

Invalid choices are removed early.

Therefore:

iv. Real-World Challenges and Best Strategy

Challenges include:

Hundreds of courses.

Thousands of students.

Limited halls.

Faculty availability.

Student conflicts.

Special accommodations.

Multiple exam sessions.

Last-minute changes.

A simple brute-force search becomes extremely expensive because the number of possible timetables grows exponentially.

Recommended Strategy

Use:

MRV (Minimum Remaining Values) selects the variable with the smallest remaining domain.

This follows the principle:

Solve the most constrained exam first.

This combination provides significantly better scalability than basic backtracking alone.

5. Strategic Game AI – Minimax and Alpha-Beta Pruning

i. Minimax

A strategic game can be represented as a game tree.

There are two players:

MAX = AI player

MIN = opponent

MAX tries to maximise the score.

MIN tries to minimise it.

Example:



At MIN nodes:

At MAX:

Therefore, MAX chooses A.

ii. Computational Challenge

If:

Branching factor = b

Search depth = d

then the number of nodes is approximately:

Suppose:

Then:

nodes may need to be evaluated.

If:

then:

nodes.

This makes exhaustive Minimax extremely expensive.

iii. Alpha-Beta Pruning

Alpha-Beta pruning removes branches that cannot affect the final decision.

Two values are maintained:

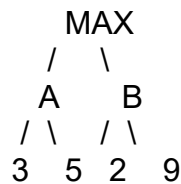
Alpha

Beta

Pruning occurs when:

Detailed Example

Consider:



First evaluate A:

Therefore MAX has:

Now examine B.

First child gives:

At B, MIN already has a value of 2.

Since MAX already has 3 from A:

Therefore:

So the remaining branch containing 9 does not need to be evaluated.

This is called pruning.

Benefit

The final answer remains the same, but fewer nodes are searched.

iv. Evaluation Function

In a real-time strategy game, searching until the end of the game may be impossible.

Therefore, the AI uses an evaluation function.

For example:

where:

- ☐ = military strength
- ☐ = territory controlled
- ☐ = resources
- ☐ = production capacity
- ☐ = defensive strength
- ☐ = health/unit survival.

The weights represent importance.

Example

Suppose:

and simplified evaluation:

Then:

A higher score means a better state for MAX.

v. Recommended Real-Time Strategy

For a real-time game, I recommend:

with:

Good evaluation function

Move ordering

Iterative deepening

Time limit

Transposition tables

Why?

Alpha-Beta pruning reduces unnecessary search.

Depth limiting prevents the AI from spending too much time searching.

Iterative deepening allows the AI to produce the best answer found before the time limit.

For example:

Time = 100 ms

Depth 1 → Search

Depth 2 → Search

Depth 3 → Search

Depth 4 → Search

Time nearly over



Return best Depth-4 decision

Overall Rubric-Oriented Summary

Scenario

Main AI Technique

Key Formula/Concept

Best Approach

Emergency Drone

Greedy BFS / A*



A + dynamic replanning*

Smart Traffic

Optimisation



GA + local optimisation

Mars Rover

Online Search

Sense → Plan → Act

Online informed search

Exam Timetable

CSP



Backtracking + MRV + Forward Checking

Game AI

Minimax



Alpha-Beta + depth-limited Minimax

Important calculation points to remember

A*:

Greedy Best-First:

Simulated Annealing:

CSP:

Minimax:

Alpha-Beta pruning:

Search complexity:

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand